

Lecture 19: Two Pointer in C++

PATTERN OVERVIEW

Two Pointers tab use hota hai jab array sorted ho aur hume pair ya triplet find karna ho. Is lecture me hum **2Sum se lekar K-Sum tak** ka pattern samajhne wale hain. Har problem ko brute force se shuru karke, observation ke through optimal tak pahunchayenge.



Problem 1 — Two Sum II (Sorted Array)

PROBLEM STATEMENT

Ek **sorted array** diya gaya hai aur ek **target**. Tumhe do aise numbers find karne hain jinka sum exactly target ke barabar ho. Return karna hai **1-based indices**.

Example: numbers = [2,7,11,15], target = 9 → Output: [1,2] kyunki 2 + 7 = 9

Core Intuition

FIRST PRINCIPLES

Imagine ek line me log shortest se tallest order me khade hain. Tumhe do log chahiye jinki height ka total exactly X ho. **Sorted property** ka fayda uthaoge ya har pair check karoge? Yahi problem ka essence hai.

BRUTE FORCE $O(N^2)$

APPROACH

Har element ke saath har doosra element check karo. Do nested loops se har pair ka sum nikalo.

```
// Brute Force:  $O(N^2)$  Time,  $O(1)$  Space
class Solution {
public:
    vector<int> twoSum(vector<int>& numbers, int target) {
        int n = numbers.size();
        for(int i = 0; i < n; i++){
            for(int j = i + 1; j < n; j++){
                if(numbers[i] + numbers[j] == target){
                    return {i + 1, j + 1};
                }
            }
        }
        return {};
    }
};
```

KEY INSIGHT

Array sorted hai. **Left = smallest, Right = largest**. Current sum nikalo:

- **Sum < Target** → Left++ (sum badhana hai, chhote ko badao)
- **Sum > Target** → Right-- (sum kam karna hai, bade ko ghatao)
- **Sum == Target** → Answer mil gaya

```
// Optimized: Two Pointers - O(N) Time, O(1) Space
class Solution {
public:
    vector<int> twoSum(vector<int>& numbers, int target) {
        int left = 0;
        int right = numbers.size() - 1;

        while (left < right) {
            int sum = numbers[left] + numbers[right];

            if (sum == target) {
                return {left + 1, right + 1};
            }
            else if (sum < target) {
                left++;
            }
            else {
                right--;
            }
        }
        return {};
    }
};
```

Step-by-Step Dry Run

DRY RUN VISUALIZATION

Input: numbers = [2, 7, 11, 15] Target = 9

STEP 1 Array: **2** **7** **11** **15** L=0 ↔ R=3 Sum = 2 + 15 = 17 17 > 9
→ Sum bada hai, Right--

STEP 2 Array: **2** **7** **11** **15** L=0 ↔ R=2 Sum = 2 + 11 = 13 13 > 9
→ Sum bada hai, Right--

STEP 3 Array: **2** **7** **11** **15** L=0 ↔ R=1 Sum = 2 + 7 = 9 9 = 9
→ **MATCH! Return [1, 2]**

Result: [1, 2] (1-based indexing)

COMPLEXITY ANALYSIS

Approach	Time	Space	When to Use
Brute Force	$O(N^2)$	$O(1)$	Only if array unsorted & no extra space
HashMap	$O(N)$	$O(N)$	Unsorted array, space allowed
Two Pointers	$O(N)$	$O(1)$	Sorted array — BEST

COMMON MISTAKES

- ✖ **Unsorted array** par Two Pointers laga dena — ye sirf sorted array par kaam karta hai.
- ✖ **0-based indices** return karna — problem 1-based maangti hai.
- ✖ Sum bada hone par **left++** kar dena — ye sum aur bada kar dega. Rule: Sum > Target → Right--

INTERVIEW DISCUSSION

Q: HashMap se bhi $O(N)$ ho jayega, phir Two Pointers kyu?

HashMap $O(N)$ time deta hai lekin **$O(N)$ extra space** leta hai. Yahan array sorted hai, isliye Two Pointers se $O(N)$ time + $O(1)$ space achieve kar sakte hain. Interview me ye preferred solution hai.

Problem 2 — Triplet With Given Sum

PROBLEM STATEMENT

Ek integer array arr diya gaya hai size N ka aur ek target. Determine karo ki kya **teen distinct elements** hain jinka sum exactly target ke barabar ho. Return true ya false.

Example: arr = [8,4,1,6,10], target = 15 → Output: true (1 + 4 + 10 = 15)

First Thought

BRAINSTORMING

Teen numbers choose karne hain. Natural thought: **har possible triplet check kar lo**. Ye galat nahi hai — yehi first thought honi chahiye interview me.

BRUTE FORCE **$O(N^3)$**

```
// Brute Force: 3 nested loops -  $O(N^3)$ 
bool hasTripletSum(vector<int>& arr, int target) {
    int n = arr.size();
    for(int i = 0; i < n; i++){
        for(int j = i + 1; j < n; j++){
            for(int k = j + 1; k < n; k++){
                if(arr[i] + arr[j] + arr[k] == target)
                    return true;
            }
        }
    }
    return false;
}
```

THE BIG OBSERVATION

KEY INSIGHT

Equation: $a + b + c = \text{target}$. Teen unknown hain. Agar **a ko fix** kar dein, to equation ban jati hai:

$$b + c = \text{target} - a$$

Ab sirf **2 unknown** bache — ye **Two Sum** ban gaya! Sorted array par Two Pointers se $O(N)$ me solve kar sakte hain.

OPTIMIZED ALGORITHM $O(N^2)$ TIME · $O(1)$ SPACE

WORKFLOW

Sort Array → Fix One Element (i) → $\text{newTarget} = \text{target} - \text{arr}[i]$ → Apply Two Pointers on remaining array → Found? Return true : Continue

```
// Optimized: Sort + Two Pointers -  $O(N^2)$  Time,  $O(1)$  Space
bool hasTripletSum(vector<int> &arr, int target) {
    sort(arr.begin(), arr.end());
    int n = arr.size();

    for(int i = 0; i < n - 2; i++) {
        int left = i + 1;
        int right = n - 1;
        int newTarget = target - arr[i];

        while(left < right) {
            int sum = arr[left] + arr[right];

            if(sum == newTarget) {
                return true;
            }
            else if(sum < newTarget) {
                left++;
            }
            else {
                right--;
            }
        }
    }
    return false;
}
```

Complete Dry Run

DRY RUN

Input: arr = [8, 4, 1, 6, 10] Target = 15 **After Sort:** [1, 4, 6, 8, 10]

ITERATION 1 Fix i = 0 → **1** is fixed Need = 15 - 1 = **14** Subarray to search:
[4, 6, 8, 10] **4** **6** **8** **10** L=1 ↔ R=4 Sum = 4 + 10 = **14** 14 = 14
→ **MATCH! Return true**

Triplet Found: 1 + 4 + 10 = 15 → true

COMPLEXITY ANALYSIS

Approach	Time	Space
Brute Force (3 loops)	$O(N^3)$	$O(1)$
HashMap (for each i)	$O(N^2)$	$O(N)$
Sort + Two Pointers	$O(N^2)$	$O(1)$

Sorting $O(N \log N)$ + Outer loop $O(N)$ × Two Pointers $O(N)$ = $O(N^2)$ overall

INTERVIEW DISCUSSION

Q: Tumne ek element fix hi kyun kiya?

Original equation $a + b + c = \text{target}$ me 3 unknown the. Ek element fix karne se equation $b + c = \text{target} - a$ ban gayi. Isse problem Two Sum me reduce ho gayi, jise sorted array par Two Pointers ki madad se linear time me solve kiya ja sakta hai.

Problem 3 — Quadruplet With Given Sum

PROBLEM STATEMENT

Ek integer array arr size N ka aur ek target diya gaya hai. Determine karo ki kya **char distinct elements** hain jinka sum exactly target ke barabar ho.

Example: arr = [3,7,1,2,8,4,5], target = 20 → Output: true (2 + 3 + 7 + 8 = 20)

First Thought

BRAINSTORMING

Char numbers choose karne hain. Natural instinct: **4 nested loops** se har quadruplet check kar lo. Lekin $O(N^4)$ bahut slow hai.

BRUTE FORCE $O(N^4)$

```
// Brute Force: 4 nested loops -  $O(N^4)$ 
bool find4Numbers(vector<int>& arr, int target) {
    int n = arr.size();
    for(int i = 0; i < n; i++){
        for(int j = i + 1; j < n; j++){
            for(int k = j + 1; k < n; k++){
                for(int l = k + 1; l < n; l++){
                    if(arr[i] + arr[j] + arr[k] + arr[l] == target)
                        return true;
                }
            }
        }
    }
    return false;
}
```

THE BIG OBSERVATION

KEY INSIGHT

Equation: $a + b + c + d = \text{target}$. 4 unknown hain. Ek element fix karne se bhi 3 unknown bachte hain — abhi bhi costly.

Do elements fix karo: a aur b ko fix karne se equation ban jati hai:

$$c + d = \text{target} - a - b$$

Ab sirf **2 unknown** — ye Two Sum ban gaya! Sorted array par Two Pointers se solve.

OPTIMIZED ALGORITHM $O(N^2)$ TIME · $O(1)$ SPACE

WORKFLOW

Sort Array → Fix First Element (i) → Fix Second Element (j) → $\text{newTarget} = \text{target} - \text{arr}[i] - \text{arr}[j]$ → Apply Two Pointers on remaining subarray → Found? Return true : Continue

// Optimized: Sort + Two Pointers - $O(N^3)$ Time, $O(1)$ Space

```
bool find4Numbers(vector<int>& A, int X) {
    sort(A.begin(), A.end());
    int n = A.size();

    for(int i = 0; i < n - 3; i++) {
        for(int j = i + 1; j < n - 2; j++) {
            int left = j + 1;
            int right = n - 1;
            int newTarget = X - A[i] - A[j];

            while(left < right) {
                int sum = A[left] + A[right];

                if(sum == newTarget) {
                    return true;
                }
                else if(sum < newTarget) {
                    left++;
                }
                else {
                    right--;
                }
            }
        }
    }
    return false;
}
```

Complete Dry Run

DRY RUN

Input: arr = [3, 7, 1, 2, 8, 4, 5] Target = 20 After Sort: [1, 2, 3, 4, 5, 7, 8]

ITER 1 Fix i=0 → **1** Fix j=1 → **2** Need = $20 - 1 - 2 = 17$ Subarray: [3, 4, 5, 7, 8]

	3	4	5	7	8	L=2 ↔ R=6	Sum = 3 + 8 = 11 (11 < 17)
→	L++	3	4	5	7	8	L=3 ↔ R=6 Sum = 4 + 8 = 12 (12 < 17)
→	L++	3	4	5	7	8	L=4 ↔ R=6 Sum = 5 + 8 = 13 (13 < 17)
→	L++	3	4	5	7	8	L=5 ↔ R=6 Sum = 7 + 8 = 15 (15 < 17)
→ No pair found							

ITER 2 Fix i=1 → **2** Fix j=2 → **3** Need = $20 - 2 - 3 = 15$ Subarray: [4, 5, 7, 8]

	4	5	7	8	L=3 ↔ R=6	Sum = 4 + 8 = 12 (12 < 15) →
L++	4	5	7	8	L=4 ↔ R=6	Sum = 5 + 8 = 13 (13 < 15) →
L++	4	5	7	8	L=5 ↔ R=6	Sum = 7 + 8 = 15 (15 = 15) →
MATCH!						

Quadruplet Found: 2 + 3 + 7 + 8 = 20 → true

COMPLEXITY ANALYSIS

Approach	Time	Space
Brute Force (4 loops)	$O(N^4)$	$O(1)$
Sort + Two Pointers	$O(N^3)$	$O(1)$

Sorting $O(N \log N)$ + Outer $O(N)$ × Inner $O(N)$ × Two Pointers $O(N)$ = $O(N^3)$

INTERVIEW DISCUSSION

Q: Tumne do elements hi fix kyun kiye?

Original equation $a + b + c + d = \text{target}$ me 4 unknowns the. Maine a aur b ko fix kiya, jisse equation $c + d = \text{target} - a - b$ ban gayi. Ab problem Two Sum me reduce ho gayi, jise sorted array par Two Pointers se linear time me solve kar sakte hain. Isliye overall complexity $O(N^3)$ ho gayi.

Problem 4 — Pair With Given Difference

PROBLEM STATEMENT

Ek integer array arr size N ka aur ek integer diff diya gaya hai. Determine karo ki kya **do distinct elements** hain jinka difference exactly diff ke barabar ho. Return 1 (exists) ya 0 (not exists).

Example: arr = [5,20,3,2,50,80], diff = 78 → Output: 1 (80 - 2 = 78)

First Thought

BRAINSTORMING

Do numbers chahiye jinka difference given value ho. Natural thought: **har pair check kar lo** using nested loops.

BRUTE FORCE $O(N^2)$

```
// Brute Force:  $O(N^2)$  Time,  $O(1)$  Space
int pairWithGivenDifference(vector<int>& arr, int diff){
    int n = arr.size();
    for(int i = 0; i < n; i++){
        for(int j = i + 1; j < n; j++){
            if(abs(arr[i] - arr[j]) == diff)
                return 1;
        }
    }
    return 0;
}
```

THE BIG OBSERVATION

KEY INSIGHT

Equation: **larger - smaller = diff**. Agar array ko **sort** kar dein, to hamesha $\text{arr}[\text{right}] \geq \text{arr}[\text{left}]$ rahega. Ab difference nikalna easy ho gaya — **Two Pointers** directly apply kar sakte hain.

OPTIMIZED ALGORITHM $O(N \log N)$ TIME · $O(1)$ SPACE

WORKFLOW

Sort Array $\rightarrow$ start = 0, end = 1 $\rightarrow$ currentDiff = $\text{arr}[\text{end}] - \text{arr}[\text{start}]$

- **currentDiff < diff** $\rightarrow$ end++ (bada number chahiye, gap badhana hai)
- **currentDiff > diff** $\rightarrow$ start++ (chhota number bada karo, gap kam karo)
- **currentDiff == diff** $\rightarrow$ Pair mil gaya, return 1

Note: start == end hone par end++ karna padega taaki same element compare na ho.

```
// Optimized: Sort + Two Pointers -  $O(N \log N)$  Time,  $O(1)$  Space
int pairWithGivenDifference(vector<int>& arr, int diff) {
    sort(arr.begin(), arr.end());
    int n = arr.size();
    int start = 0;
    int end = 1;

    while (end < n) {
        if (start == end) {
            end++;
            continue;
        }

        int currentDiff = arr[end] - arr[start];

        if (currentDiff == diff) {
            return 1;
        }
        else if (currentDiff < diff) {
            end++;
        }
        else {
            start++;
        }
    }
    return 0;
}
```

Complete Dry Run

DRY RUN

Input: arr = [5, 20, 3, 2, 50, 80] Diff = 78 **After Sort:** [2, 3, 5, 20, 50, 80]

STEP 1 Array: 2 3 5 20 50 80 S=0 ↔ E=1 Diff = 3 - 2
= 1 1 < 78 → Diff chhota hai, End++

STEP 2 Array: 2 3 5 20 50 80 S=0 ↔ E=2 Diff = 5 - 2
= 3 3 < 78 → Diff chhota hai, End++

STEP 3 Array: 2 3 5 20 50 80 S=0 ↔ E=3 Diff = 20 - 2
= 18 18 < 78 → Diff chhota hai, End++

STEP 4 Array: 2 3 5 20 50 80 S=0 ↔ E=4 Diff = 50 - 2
= 48 48 < 78 → Diff chhota hai, End++

STEP 5 Array: 2 3 5 20 50 80 S=0 ↔ E=5 Diff = 80 - 2
= 78 78 = 78 → **MATCH! Return 1**

Pair Found: 80 - 2 = 78 → 1

COMPLEXITY ANALYSIS

Approach	Time	Space
Brute Force (2 loops)	$O(N^2)$	$O(1)$
HashSet	$O(N)$	$O(N)$
Sort + Two Pointers	$O(N \log N)$	$O(1)$

INTERVIEW DISCUSSION

Q: Difference chhota hai to end hi kyun badhaya?

Sorted array me arr[end] hamesha arr[start] se bada ya barabar hota hai. Agar current difference target se chhota hai, to difference badhane ke liye bade element ko aur bada banana hoga, isliye end++ karte hain. Agar difference target se bada hai, to chhote element ko bada karna hoga taaki gap kam ho, isliye start++ karte hain.



Pattern Transfer — The K-Sum Framework

UNIFIED PATTERN

Jitne numbers choose karne hain, unme se **(K - 2) numbers fix karo** , baaki 2 numbers ko Two Pointers se solve karo. Yehi first-principles thinking hai.

K Numbers	Fix (K-2)	Solve With	Complexity
2 Sum	Fix 0	Two Pointers	$O(N)$
3 Sum	Fix 1	Two Pointers	$O(N^2)$
4 Sum	Fix 2	Two Pointers	$O(N^3)$
K Sum	Fix (K-2)	Two Pointers	$O(N^{(K-1)})$

KEY TAKEAWAYS

- **Two Pointers** sirf sorted array par kaam karta hai. Pehle sort karo, phir apply karo.
- **Sum problems** me: $\text{Sum} < \text{Target} \rightarrow \text{Left}++$ | $\text{Sum} > \text{Target} \rightarrow \text{Right}--$
- **Difference problems** me: $\text{Diff} < \text{Target} \rightarrow \text{End}++$ | $\text{Diff} > \text{Target} \rightarrow \text{Start}++$
- **K-Sum** ka universal rule: (K-2) elements fix karo, baaki Two Pointers se.
- HashMap $O(N)$ space leta hai; Two Pointers $O(1)$ space — interview me preferred.

BEST PRACTICES

- Hamesha **edge cases** check karo: empty array, single element, duplicate values.
- Two Pointers me **while(left < right)** condition ka dhyan rakho — equal hone par loop band.
- 1-based vs 0-based indexing ka confusion mat karo — problem statement padh ke decide karo.
- Interviewer se pooch lo: "Can I modify the array?" — agar nahi, to HashMap socho.



◆ QUICK REVISION CHEATSHEET ◆

Two Sum II (Sorted)

Left=0, Right=n-1

$\text{Sum} < T \rightarrow L++$ | $\text{Sum} > T \rightarrow R--$

Time: $O(N)$ | Space: $O(1)$

Triplet Sum (3Sum)

Sort $\rightarrow$ Fix i $\rightarrow$ Two Pointers on [i+1, n-1]

Need = target - arr[i]

Time: $O(N^2)$ | Space: $O(1)$

Quadruplet Sum (4Sum)

Sort $\rightarrow$ Fix i, j $\rightarrow$ Two Pointers on [j+1, n-1]

Need = target - arr[i] - arr[j]

Time: $O(N^3)$ | Space: $O(1)$

Pair With Difference

Sort $\rightarrow$ start=0, end=1

$\text{Diff} < T \rightarrow E++$ | $\text{Diff} > T \rightarrow S++$

Time: $O(N \log N)$ | Space: $O(1)$

Golden Rule (Sum)

Sorted + Pair Find $\rightarrow$ Two Pointers

$\text{Sum} < \text{Target} \rightarrow \text{Left}++$

$\text{Sum} > \text{Target} \rightarrow \text{Right}--$

Golden Rule (Diff)

Sorted + Diff Find $\rightarrow$ Two Pointers

$\text{Diff} < \text{Target} \rightarrow \text{End}++$

$\text{Diff} > \text{Target} \rightarrow \text{Start}++$

Common Mistakes

- × Unsorted array pe Two Pointers
- × 0-based return (1-based maanga ho)
- × Sum bada hone pe L++

Interview Questions

- HashMap vs Two Pointers? $\rightarrow$ Space
- K-Sum ka generalization? $\rightarrow$ Fix K-2
- Diff problem me start==end? $\rightarrow$ Skip

MADE BY HARSHAL CHAUHAN